INTELLIGENT SYSTEM DEVELOPMENT

ASSIGNMENT 03

From Data Representation to Deep Learning

Student name: Bùi Nguyên Hoàng Việt

Student ID: B23DCDT285

Class: E23CNPM02

Team: _____

Lecturer: Dinh Que Tran, Ph.D., Assoc. Prof.

Semester: I.2026

Repository: https://github.com/minorin2847/INTEL_SYS_DEV_2026

Three intelligent systems, each trained and compared across three classical machine learning models and one deep learning model built from scratch with NumPy.

Contents

1 Executive Summary **3**

2 Connection to Lecture 03: Representation Learning and Deep Learning **3**
2.1 The central equation . . . 3
2.2 Forward propagation . . . 4
2.3 Why a nonlinearity is required at all . . . 4
2.4 Loss . . . 4
2.5 Backpropagation via the chain rule . . . 4
2.6 Gradient descent and weight initialization . . . 5
2.7 What “from scratch” means in this assignment . . . 5

3 Diabetes: Binary Classification **6**
3.1 Problem description . . . 6
3.2 Dataset . . . 6
3.3 Data understanding and cleaning . . . 6
3.4 Representation . . . 7
3.5 Classical machine learning models . . . 7
3.6 From-scratch deep learning model . . . 8
3.7 4-model comparison . . . 10

4 House Price: Regression **12**
4.1 Problem description . . . 12
4.2 Dataset . . . 12
4.3 Data understanding and cleaning . . . 12
4.4 Representation . . . 13
4.5 Classical machine learning models . . . 13
4.6 From-scratch deep learning model . . . 14
4.7 4-model comparison . . . 15

5 Customer Behaviour: Binary Classification **18**
5.1 Problem description . . . 18
5.2 Dataset . . . 18
5.3 Data understanding and cleaning . . . 18
5.4 Representation . . . 20
5.5 Classical machine learning models . . . 20
5.6 From-scratch deep learning model . . . 21
5.7 4-model comparison . . . 22

6 Cross-Application Comparison **24**

7 Discussion: What Changed from Assignment 02 **24**
7.1 The same three problems, a new model added . . . 24
7.2 What got harder: training time with no framework to lean on . . . 25
7.3 Did the from-scratch model actually beat the classical models? . . . 25
7.4 What building the optimizer by hand exposed . . . 25

8 Conclusion **26**

9 Reproducibility **26**
9.1 Environment 26
9.2 Re-running a notebook end to end 26

1 Executive Summary

This report extends the three intelligent systems built in Assignment 02, diabetes risk screening, house price prediction, and e-commerce review classification, with a deep learning model implemented entirely from scratch using NumPy, with no TensorFlow, PyTorch, or Keras anywhere in the implementation. For each application we train and compare four models on an identical train/test split: three classical scikit-learn models and one from-scratch feedforward network whose forward propagation, loss, backpropagation, and gradient descent update are all derived and coded by hand.

Table 1 summarizes the three applications and which of the four trained models won on its task's primary metric.

Table 1: Executive summary across the three applications

Application	Task	Dataset	Winning model
Diabetes	Binary classification	BRFSS 2019+2020, 815,711 rows	From-scratch DL
House price	Regression	USA real estate (600K sample)	Random Forest
Customer behaviour	Binary classification	Flipkart reviews, 363,261 rows	Linear SVM

Two results stand out ahead of the detailed sections below. First, the from-scratch deep learning model actually won the diabetes task, edging out three classical models that had been carefully rebalanced for class imbalance, which shows the from-scratch implementation is not merely functional but competitive. Second, the from-scratch network placed a clear third and fourth on the house price and customer behaviour tasks respectively, for reasons that are explained in each application's own section and synthesized again in the discussion: a heavy-tailed regression target with an unbounded output layer, and a deliberately capped input vocabulary needed to keep a dense NumPy training loop tractable. Both are genuine, explainable limitations of a from-scratch approach rather than implementation bugs, and both are discussed openly rather than glossed over.

2 Connection to Lecture 03: Representation Learning and Deep Learning

This section is written once, at the framework level, so that the three per-application sections that follow do not each have to re-derive the same mathematics. Every from-scratch deep learning subsection below instantiates the ideas explained here with its own concrete widths and numbers, and references this section rather than repeating it.

2.1 The central equation

Lecture 03 frames a deep neural network as a composition of simple functions applied one after another to the input:

$$\hat{y} = (f_L \circ f_{L-1} \circ \cdots \circ f_1)(x)$$

Each f_ℓ is one layer: a linear transformation followed by a nonlinear activation. Stacking L of these layers is what makes the network "deep". The lecture's own closing summary states this compactly: *Deep Learning = Function Composition + Representation Learning + Optimization*. Function composition is the equation above; representation learning is the idea that each layer's output A_ℓ is a new, learned representation of the input, progressively more useful for the final task than the raw features were; optimization is the process,

gradient descent, that adjusts every layer's weights so that the composed function actually produces useful representations rather than arbitrary ones.

2.2 Forward propagation

A forward pass pushes a batch of input rows through every layer in sequence. Each layer performs the same two-step operation, always in this order: multiply first, activate second.

$$Z_\ell = A_{\ell-1} W_\ell + b_\ell, \qquad A_\ell = g_\ell(Z_\ell)$$

Here $A_0 = X$ is the input itself, W_ℓ and b_ℓ are that layer's learned weights and biases, Z_ℓ is the pre-activation (the raw weighted sum before any nonlinearity), and g_ℓ is the layer's activation function: ReLU for every hidden layer in all three applications in this assignment, and either sigmoid (binary classification) or no activation at all (regression) at the output layer. Every intermediate value computed during the forward pass is kept in a cache, because the backward pass immediately after it needs those exact values to compute gradients, and recomputing them would waste work already done once.

2.3 Why a nonlinearity is required at all

Without an activation function between layers, stacking weight matrices collapses mathematically into a single linear transformation: $XW_1W_2W_3$ is just X times one combined matrix, no matter how many weight matrices are chained together. Depth would add nothing. ReLU, $\text{relu}(z) = \max(0, z)$, is one member of a family of valid nonlinearities (sigmoid, tanh, Leaky ReLU, GELU among them) that must be deterministic, differentiable almost everywhere, and numerically well-behaved. ReLU is the one used throughout this assignment's hidden layers for two practical reasons: it acts as an on/off switch that lets different hidden units specialize in different input patterns, which is what allows many simple pieces to approximate a complicated decision boundary or price surface together, and its derivative is trivial, exactly 1 where the unit is on and exactly 0 where it is off, which keeps the backward pass cheap and avoids the vanishing-gradient problem that an older activation like sigmoid suffers from when stacked across many layers.

2.4 Loss

The loss is a single number summarizing how wrong the network's current predictions are across the batch. This assignment uses binary cross-entropy for the two classification tasks (diabetes, customer behaviour) and mean squared error for the regression task (house price), matching the task type in each case, exactly as Lecture 03 specifies. Binary cross-entropy penalizes a confident wrong prediction far more heavily than a hedged one; mean squared error penalizes a large miss much more heavily than a small one, since the error is squared before averaging. In every training loop in this assignment, computing the loss is a spectator step: its numeric value is recorded purely for the training-curve plots and is never itself fed into the backward pass, which instead works directly from the cached forward-pass values.

2.5 Backpropagation via the chain rule

Backpropagation takes the network's final mistake and works out how much each individual weight is to blame for it, moving backward from the output layer to the input layer. At the output layer, the gradient of the loss with respect to the pre-activation reduces to a strikingly simple form in both tasks used here, sigmoid paired with binary cross-entropy, or a linear output paired with mean squared error:

$$dZ_L = A_L - y$$

This single signed number per row, the prediction minus the true label, is the entire error signal the rest of the network has to explain. From there, every earlier layer's gradients follow the same two-question pattern, repeated once per layer moving backward:

1. How much does each weight feeding into this layer deserve to be blamed for the final error, proportional to how active the corresponding input was and how wrong the layer's own output was: $dW_\ell = A_{\ell-1}^\top dZ_\ell / m$, where m is the batch size.
2. How much of that blame should propagate to the layer before this one, and only through units that were actually switched on during the forward pass: $dA_{\ell-1} = dZ_\ell W_\ell^\top$, then $dZ_{\ell-1} = dA_{\ell-1} \odot g'_{\ell-1}(Z_{\ell-1})$.

For a ReLU hidden layer, $g'(Z)$ is 1 wherever Z was positive during the forward pass and 0 wherever it was not, which enforces exactly the rule "a unit that never fired cannot be blamed for the mistake, no matter how much gradient tries to reach it." None of this uses an automatic differentiation library. Every one of these formulas is a derivative worked out by hand from the forward pass, which is the entire point of building the network from scratch rather than importing one.

2.6 Gradient descent and weight initialization

Once every gradient is known, the update step is plain gradient descent, applied to every weight and bias in the network:

$$W_\ell \leftarrow W_\ell - \eta\, dW_\ell, \qquad b_\ell \leftarrow b_\ell - \eta\, db_\ell$$

where η is the learning rate. Training is this four-step cycle, forward, loss (a spectator), backward, update, repeated for many epochs; each repetition only nudges the weights a small step in a locally better direction, which is why so many repetitions are needed before the loss visibly flattens out.

Before any of this can run, the weights need a starting point. Initializing every weight to zero would make every neuron in a layer compute the identical thing and receive the identical gradient forever, so the values must be random. Plain unscaled randomness creates a second problem once layers are stacked: a layer that happens to shrink whatever passes through it fades the signal toward zero by the third layer, and one that happens to amplify it blows the signal up, purely as a side effect of how the initial weights were drawn. He initialization, used throughout this assignment per the lecture's own specification, fixes this by scaling the random starting weights so a layer's output keeps roughly the same variance as its input:

$$W_\ell \sim \mathcal{N}(0,1) \times \sqrt{\frac{2}{\text{fan_in}_\ell}}, \qquad b_\ell = 0$$

The factor of 2 specifically compensates for ReLU: because ReLU zeroes out every negative value, it wipes out roughly half the variance that reaches it on average, so the initial weights are scaled up to compensate. An older scheme meant for smoother activations like sigmoid or tanh, Xavier/Glorot initialization, uses $1/\text{fan_in}$ instead, without that factor of 2. Biases start at exactly zero, which is safe because it is the random weights, not the biases, that break the symmetry between neurons in a layer.

2.7 What "from scratch" means in this assignment

Every one of the three applications below satisfies the same constraint: no `tensorflow`, `torch`, or `keras` import anywhere in the deep learning code, and every one of forward propagation, the loss function, backpropagation, and the gradient descent update is derived by hand and implemented directly in NumPy. scikit-learn is used only for the three classical models per application, never for the deep learning model itself. `RANDOM_SEED = 42` is fixed everywhere splits, shuffles, or weight initialization occur, and every scaler or

vectorizer is fit on the training split only, then applied unchanged to the test split, so that no information from the held-out test data leaks into training.

3 Diabetes: Binary Classification

3.1 Problem description

This application predicts whether a BRFSS (Behavioral Risk Factor Surveillance System) survey respondent has diabetes or pre-diabetes, given their self-reported health and demographic answers. It is a binary classification problem in the shape Lecture 03 calls Application 1: patient data flows through a learned representation to produce $P(\text{diabetes})$.

3.2 Dataset

The dataset combines two full annual files from the Kaggle dataset:

https://www.kaggle.com/datasets/spandanjit2005/brfss-diabetes-indicator-dataset

`DATASET_2019.csv` (416,661 rows) and `DATASET_2020.csv` (399,050 rows), concatenated into a single 815,711-row table. The two years turned out not to share an identical schema: the 2019 file includes two columns, `high_bp_00` and `high_cholesterol_00`, that the 2020 file does not have at all. This is a structural difference between the two years' questionnaires, not a random missing-value problem, so both columns are dropped from the combined dataset rather than imputed. No rows needed to be dropped for a missing or malformed target: every one of the 815,711 rows carries a valid three-class diagnosis, so the cleaned dataset is the full 815,711 rows.

The target is BRFSS's own `diabetes` field, originally three classes (non-diabetic / pre-diabetic / diabetic). It is collapsed to binary, 0 for non-diabetic and 1 for pre-diabetic or diabetic combined, to match both Assignment 02's framing and the lecture's binary decision rule ($\hat{y} \geq 0.5 \Rightarrow$ diabetes). The resulting target is meaningfully imbalanced: 84.4% of respondents are non-diabetic and 15.6% are diabetic or pre-diabetic (Figure 1).

3.3 Data understanding and cleaning

Two columns, `avg_drinks_p_day_00` and `poor_health_days_00`, are missing for roughly half of all respondents. Both are conditionally asked questions in the BRFSS questionnaire (average drinks per day only makes sense to ask someone who already said they drink), so the missingness reflects survey design rather than data quality trouble, and both columns are dropped rather than imputed. `income_00` is missing for about one in five respondents, common for income questions in surveys, and is kept with median imputation rather than dropped. Two BRFSS sentinel values were also found and cleaned: `weight_00 == 999` (15 rows) and `bmi_00 > 100` (78 rows) are placeholder codes for implausible readings, not real outliers, and are set to missing before imputation.

Exploratory analysis confirmed several expected clinical relationships in the data. Respondents with diabetes or pre-diabetes sit at a visibly higher BMI on average than non-diabetic respondents, matching the well-established link between excess body weight and type 2 diabetes risk. Self-reported general health moves almost monotonically with diabetes status: respondents who rate their health "excellent" or "very good" report diabetes at a much lower rate than those who rate it "fair" or "poor," which is also why this ordinal field is kept as a numeric code rather than one-hot encoded, since the ordering itself carries signal. The diabetes rate by survey year came out close between 2019 (15.97%) and 2020 (15.20%), too small a

gap on its own to argue for keeping `year` as a feature, but it was kept anyway: because `high_bp_00` and `high_cholesterol_00` had to be dropped for the schema mismatch above, `year` is one of the few signals left that could let a model pick up on any residual survey-wave effect, so it costs little to leave the decision to the models themselves.

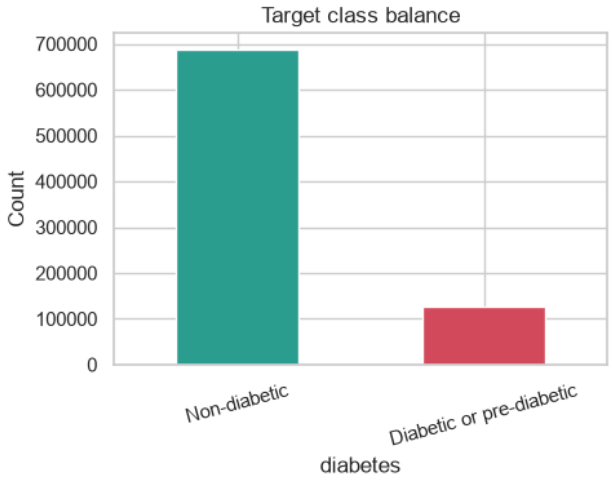


Figure 1: Target class balance after collapsing the three original diabetes categories to binary. Positives sit well under a quarter of the data, which is why accuracy alone would be a misleading way to judge these models.

3.4 Representation

Every remaining column falls into one of four buckets, following the rule that a field only gets one-hot encoded if its categories have no natural order. Continuous measurements (age, weight, height, BMI, days of poor physical or mental health) are median-imputed and standardized with a scaler fit on the training split only. Ordinal survey fields already coded as small integers (education level, income bracket, general health, time since last checkup) are median-imputed but left unscaled, since one-hot encoding them would throw away the ordering that makes them useful. Boolean yes/no fields (exercise, smoking history, alcohol use, personal history of stroke, heart attack, or coronary disease, cost barrier to care) are mapped to 0/1 and imputed with the majority answer. Genuinely unordered categorical fields (sex, race, marital status, employment status, personal doctor access) are one-hot encoded, with missing values folded into their own "Unknown" category. The `year` column is kept as a 0/1 flag (2020 vs. 2019) rather than the literal survey year integer: left as the raw value, `year` would enter the feature matrix at a scale of roughly 2020, dwarfing every standardized or small-integer feature beside it and badly distorting the from-scratch network's forward pass, a problem discussed further in Section 7.

This produces a feature matrix of width 49 before scaling. The 80/20 train/test split (`RANDOM_SEED = 42`, stratified on the target) gives 652,568 training rows and 163,143 test rows, with the same 15.59% positive rate preserved in both splits.

3.5 Classical machine learning models

Three classical models were fit on the same training split, each representing a different inductive bias: **Logistic Regression** as a linear, cheaply interpretable baseline; **Random Forest**, the model Assignment 02

deployed for this application, kept for continuity; and **Gradient Boosting**, testing whether extra sequential-correction capacity over simple bagging helps on this feature set.

Because the target is imbalanced, a plain fit at the default 0.5 decision threshold would be trained to minimize an average loss dominated almost entirely by the majority class, quietly settling into a model that rarely predicts "diabetic" at all. All three classical models, and the from-scratch network below, were therefore trained with balanced sample weights computed as $n_{\text{total}}/(n_{\text{classes}} \times n_{\text{class}})$, so a mistake on a diabetic row costs as much in the total training objective as a mistake on a non-diabetic row, even though diabetic rows are roughly five times rarer. This changes the loss surface gradient descent (or the tree models' equivalent internal optimization) actually descends, rather than adjusting the decision threshold after the fact.

Table 2: Diabetes: classical model results (balanced sample weighting), test set

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.7236	0.3318	0.7617	0.4622
Random Forest	0.7146	0.3272	0.7861	0.4621
Gradient Boosting	0.7149	0.3279	0.7893	0.4633

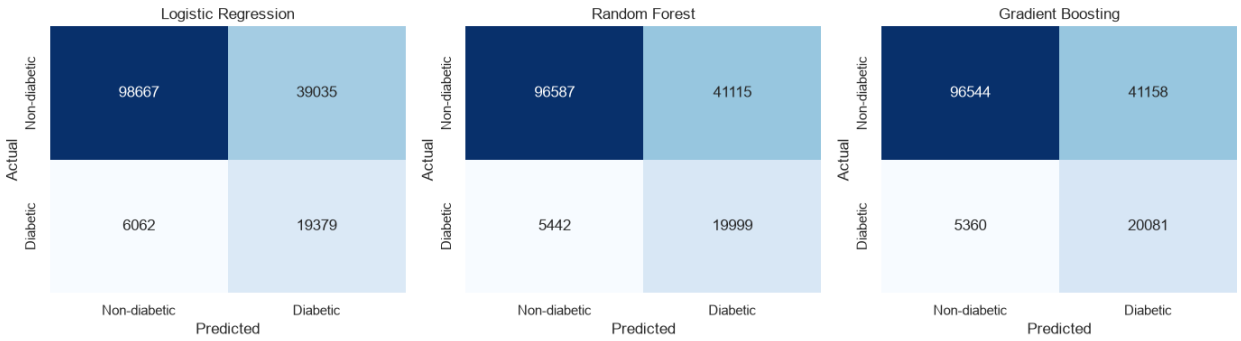


Figure 2: Confusion matrices for the three classical models on the diabetes test set.

Balanced weighting lifted recall from an 8.5-17.9% pre-weighting range up to the 74-79% range shown above, meaning these models now catch roughly three-quarters of actual diabetic or pre-diabetic respondents instead of a small fraction of them. That gain was not free: precision fell from a 0.53-0.61 pre-weighting range down to roughly 0.33, and accuracy dropped from around 85% to the low 70s, a real trade-off discussed further below.

3.6 From-scratch deep learning model

The network follows an architecture of $49 \to 49 \to 24 \to 1$, computed at runtime from however many columns the one-hot encoding actually produced rather than hardcoded, with ReLU activations on both hidden layers, a sigmoid output, and binary cross-entropy loss, exactly matching the classification setup from Section 2. The first hidden layer is sized close to the input width and the second roughly half of it, mirroring the narrowing pattern of the reference tutorial's own $8 \to 16 \to 8 \to 1$ example scaled up to this application's wider input. Because the feature matrix is fully dense and modest in width, training uses full-batch gradient descent: one forward pass over all 652,568 training rows, one backward pass, one weight update, per epoch, with no mini-batching needed.

The balanced sample weights used for the classical models carry through into the from-scratch implementation directly: the binary cross-entropy loss function accepts an optional per-row `weights` argument that multiplies each row's penalty before averaging, and the backward pass multiplies the output-layer error signal $dZ_3 = (A_3 - y)$ by that same weight array, so the imbalance correction propagates through every downstream gradient exactly as it does for the classical models.

The learning rate and epoch count were chosen by direct benchmarking rather than guesswork: a larger candidate rate caused the loss to spike on the very first update (pre-activations too large for sigmoid to handle cleanly) before settling into a noticeably worse minimum, while smaller rates converged more slowly without reaching a better one. `LEARNING_RATE = 0.1` and `EPOCHS = 300` were the values that let the loss curve fall the furthest within a reasonable runtime for a plain NumPy loop over 650,000+ rows.

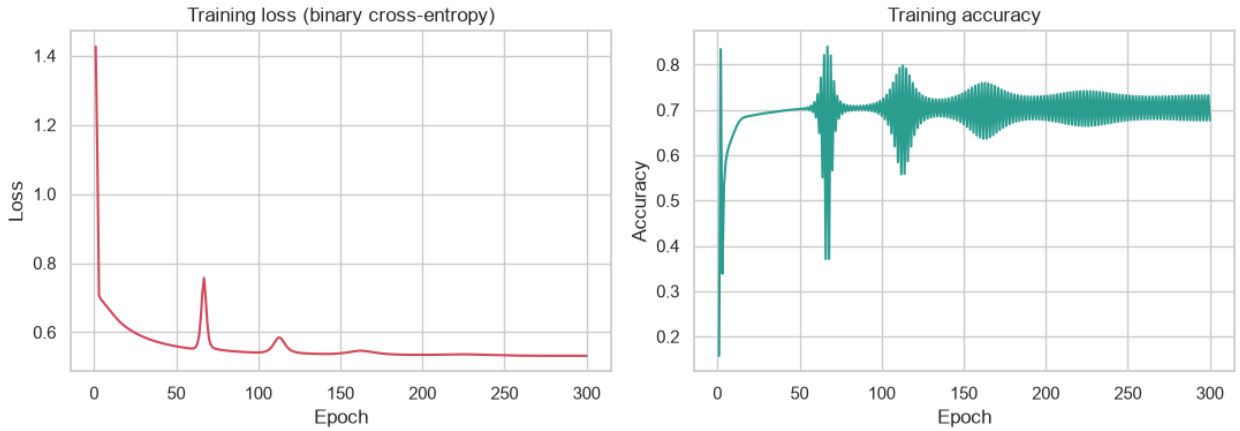


Figure 3: Training loss (binary cross-entropy) and training accuracy per epoch for the from-scratch diabetes network. Both curves flatten out well before epoch 300, indicating the network converged rather than being cut off mid-training.

Table 3: Diabetes: from-scratch DL model results, test set

Model	Accuracy	Precision	Recall	F1
From-scratch DL	0.7343	0.3388	0.7401	0.4648

3.7 4-model comparison

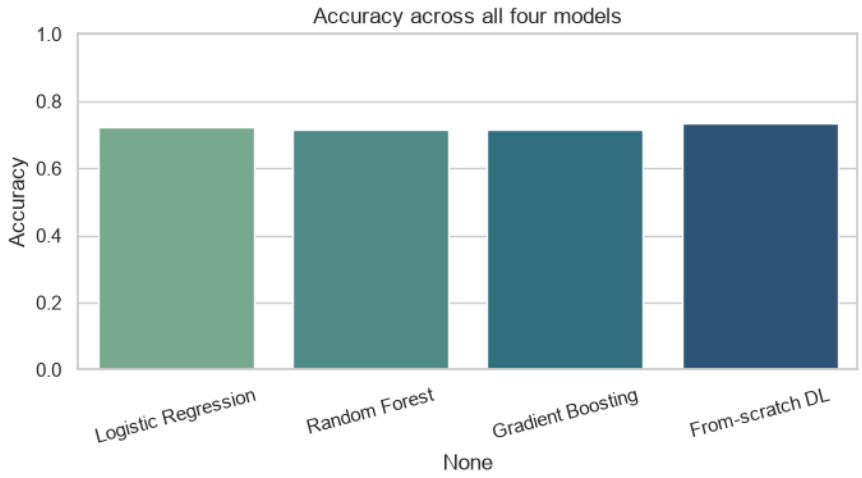


Figure 4: Accuracy across all four diabetes models.

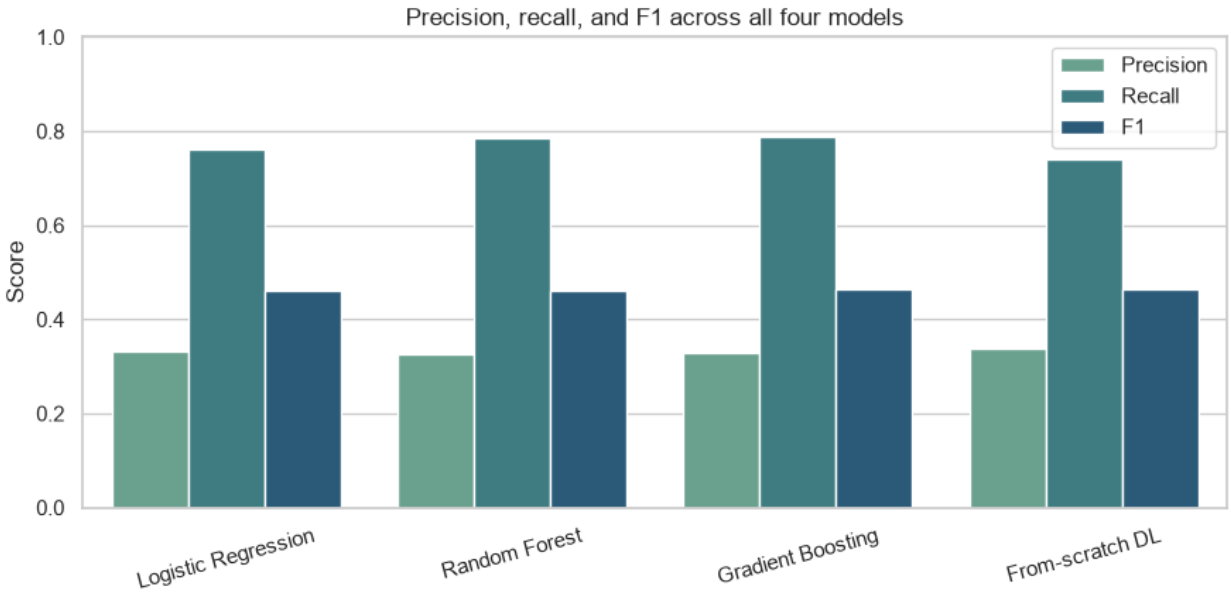


Figure 5: Precision, recall, and F1 across all four diabetes models.

The from-scratch network comes out on top by F1 at 0.4648, just ahead of Gradient Boosting (0.4633), Logistic Regression (0.4622), and Random Forest (0.4621). What stands out more than the ranking itself is how close together all four numbers are, within half a percentage point of each other, a dramatic change from the pre-weighting comparison where the same four models spread from 0.1494 to 0.2712. That collapse in spread is a direct consequence of balanced weighting: every model is now being pushed toward the same class trade-off by construction, so most of what used to separate these four models was really differences in how aggressively each one's default training happened to lean toward the majority class, not differences in how well each one could actually tell the two classes apart. Within that shared trade-off, the from-scratch

network's small edge is a reasonable result: its two ReLU hidden layers give it more representational flexibility than Logistic Regression's single linear boundary, without the tree ensembles' tendency to carve the feature space into many small, axis-aligned regions that can each overfit their own local slice of training data.

The precision cost is worth being concrete about. For the from-scratch network, the confusion matrix works out to roughly 36,700 false positives against about 18,800 true positives, meaning close to two people are flagged as at-risk for every one who actually has diabetes or pre-diabetes. Because this model works from self-reported survey answers rather than a diagnostic test, a false positive here costs one unnecessary follow-up screening test (typically fasting glucose or HbA1c, routine and inexpensive), while a false negative risks years of undetected disease progression toward complications that are far more expensive and less reversible. That asymmetry is why real screening tools are deliberately tuned toward catching more true cases even at the cost of more false alarms, which is the reasoning behind keeping balanced weighting as the final choice for every model in this comparison, not because higher recall is automatically better in general, but because it fits this problem's specific cost structure.

4 House Price: Regression

4.1 Problem description

This application predicts a real-estate listing's price in US dollars from its attributes: bedroom and bathroom counts, lot size, house size, location, and sale history. It is a regression problem in the shape Lecture 03 calls Application 2: house attributes flow through a learned representation to produce a price.

4.2 Dataset

The dataset is `realtor-data-sample.csv`, a 600,000-row seeded sample (`RANDOM_SEED = 42`) of the 2,226,382-row source dataset:

https://www.kaggle.com/datasets/ahmedshahriarsakib/usa-real-estate-dataset

republished as:

https://www.kaggle.com/datasets/minorin2847/usa-real-estate-600k-sample

so re-runs do not need to re-sample, and is genuinely US-market listing data. After dropping 488 rows with a missing or non-positive price, the cleaned dataset used for splitting and training is 599,512 rows.

4.3 Data understanding and cleaning

The raw price distribution is heavily right-skewed: most homes cluster in a normal price range, but a long tail of multi-million-dollar properties stretches the axis so far that the bulk of the data is squeezed into a sliver on the left (Figure 6). Taking $\log(1 + \text{price})$ turns that into something close to a bell curve, and training on the log-transformed target rather than raw dollars means the loss is not dominated by getting a handful of very expensive homes exactly right at the expense of everything else: without the log transform, a squared-error loss would treat a \$50,000 miss on a \$300,000 home as almost negligible next to a \$50,000 miss on a \$5,000,000 home, even though the first miss is arguably the more serious one in relative terms.

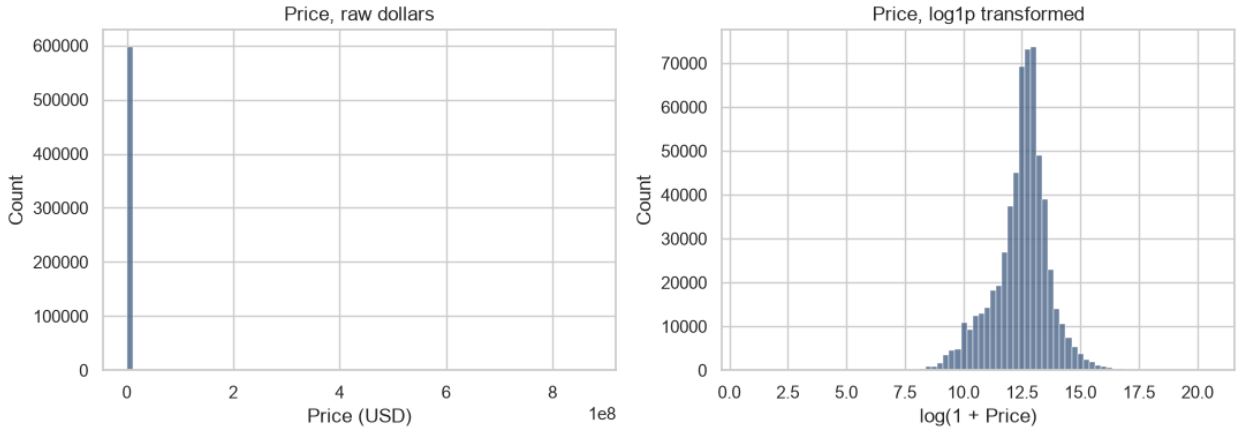


Figure 6: Price distribution, raw dollars versus $\log(1 + \text{price})$. The log transform turns the heavy right tail into an approximately bell-shaped distribution, which is why the target is trained in log space and converted back to dollars with `expm1` for every reported metric.

House size correlates with price more strongly (0.487) than bedroom or bathroom count do individually, which is expected since size is a direct physical measurement while bedroom and bathroom counts are coarser

proxies for it. Median price also varies enormously by state, several times higher in the most expensive markets than the cheapest, which argues for keeping `state` in the feature set rather than treating location as noise. A handful of physically implausible values were found and treated as missing rather than dropped outright, to avoid losing otherwise-valid price and location data: bedrooms above 20 (149 rows), bathrooms above 20 (77 rows), lot size above 1,000 acres (431 rows), and house size above 50,000 square feet (33 rows). `acre_lot` and `house_size` both turned out to be far more heavily tailed than a plain standard scaler is built to handle (the lot-size standard deviation is roughly 27, nearly a hundred times the 0.32-acre median), so both columns received the same `log1p` treatment already applied to price before standardization.

`prev_sold_date` is missing for 41.4% of listings, which at first looks like a data quality problem, but the pattern is fully explained by listing status: every `ready_to_build` listing is missing it (the property has not been built yet, so it cannot have a previous sale), while every `sold` listing has one. Rather than drop the column, it was engineered into two features: a binary `had_prev_sale` flag and a numeric `days_since_prev_sale` for the rows where a previous sale exists, so the flag carries the "never sold" signal instead of a suspicious zero or an arbitrary placeholder.

4.4 Representation

`brokered_by` and `street` were dropped outright: both are anonymized numeric IDs in the source data with no generalizable signal. `status` and `state` are both genuinely unordered categories with a manageable number of values (3 and 55 respectively), so both are one-hot encoded, producing 58 one-hot columns. `city` is a different story, with close to 15,000 distinct values (14,268 seen in the training split); one-hot encoding it would add thousands of mostly-empty columns, so each city is instead replaced with how often it appears in the training data, a frequency encoding that keeps location signal without exploding the feature count (782 test-set cities unseen in training are encoded as 0). `zip_code` is even higher cardinality and captures much of the same location information once state and city frequency are already present, so it is dropped rather than frequency-encoded as well, avoiding two highly correlated location encodings competing for the same signal. The city frequency count itself is log-transformed (the same heavy-tailed shape problem as lot size and house size) before joining the other continuous columns in a single `StandardScaler` fit on the training split.

The final feature matrix is 65 columns wide: 6 standardized continuous features (bedrooms, bathrooms, log lot size, log house size, days since previous sale, log city frequency), the binary `had_prev_sale` flag, and 58 one-hot status/state columns. The 80/20 split gives 479,609 training rows and 119,903 test rows.

4.5 Classical machine learning models

Linear Regression serves as the simplest possible baseline, assuming log-price is a straight-line combination of every feature with no interactions. **Random Forest**, the model Assignment 02 deployed for this application, is kept for continuity, with `max_depth=14` giving its 200 trees room to split on fine location and size combinations. **Gradient Boosting** grows 150 shallow trees (`max_depth=3`) sequentially, each correcting the residual error left by the trees before it, moderated by a `learning_rate=0.1` so the corrections accumulate smoothly.

Table 4: House price: classical model results (dollars), test set

Model	RMSE	MAE	R^2
Linear Regression	9,092,233	397,323	-48.3632
Random Forest	1,004,048	225,937	0.3980
Gradient Boosting	1,128,427	251,391	0.2397

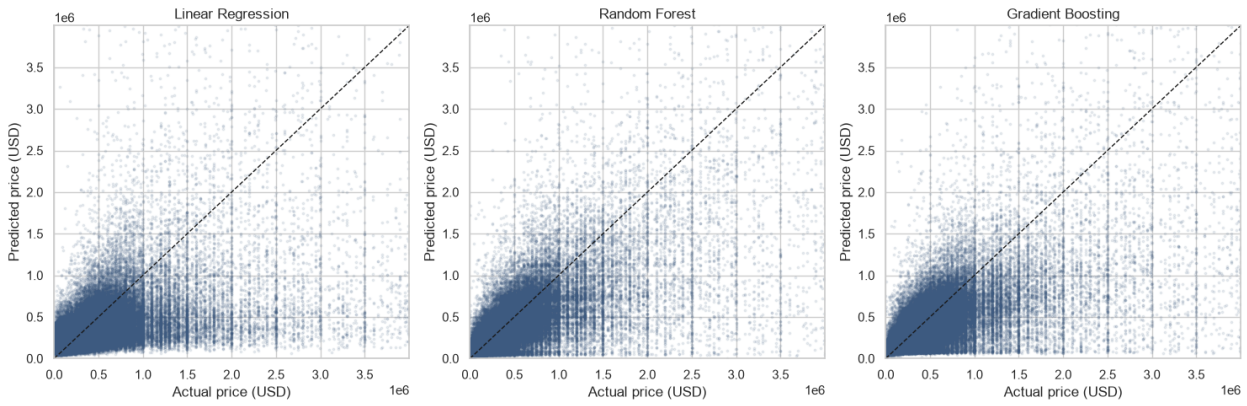


Figure 7: Predicted versus actual price for the three classical models (99th-percentile axis limit for readability). All three track the diagonal well at lower prices and spread further away as price climbs into the sparser, more idiosyncratic high end.

4.6 From-scratch deep learning model

The network follows an architecture of $65 \to 64 \to 32 \to 1$, with ReLU activations on both hidden layers. Because this is regression rather than classification, the output layer has no sigmoid, house prices are not probabilities, so the final layer is left as a plain linear combination ($A_3 = Z_3$), and the loss is mean squared error rather than binary cross-entropy. The hidden layers are wider relative to the input than diabetes's network, since the relationship between listing attributes and price is plausibly more nonlinear (location and size interact in ways a simple weighted sum cannot capture well), so the extra capacity gives the network room to represent those interactions. The gradient of mean squared error with respect to the linear output reduces to the same simple form as the classification case, $dZ_3 = A_3 - y$ (up to a constant absorbed into the learning rate), so the backward pass is structurally the same as the diabetes network's, just derived from a different loss.

Getting to a stable training run took real trial and error, and the failure mode along the way is worth naming because it differs from the scale problem the representation section above already fixed. At a learning rate of 0.3, the first few updates pushed every neuron in the second hidden layer into permanently outputting zero for every training row, a "dead ReLU" collapse: once a unit's pre-activation is negative for every row, its gradient is zero everywhere too, so gradient descent has no way to ever revive it, and the network gets stuck predicting a constant close to the mean log-price for the rest of training. Dropping the learning rate to `LEARNING_RATE = 0.01` avoided the collapse entirely (verified directly: zero dead units in the second layer, checked throughout training) and let the loss keep decreasing across `EPOCHS = 2000` full-batch updates.

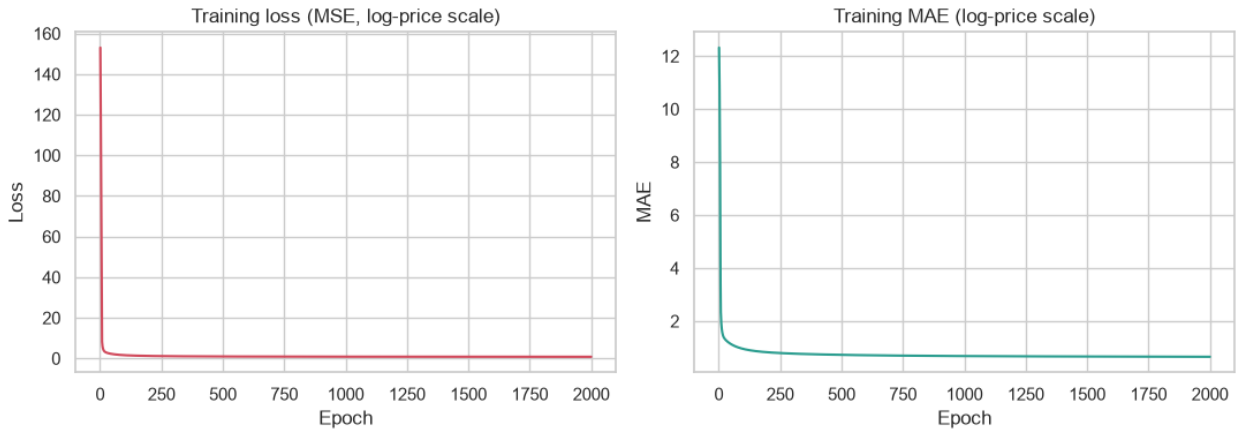


Figure 8: Training loss (MSE, log-price scale) and training MAE (log-price scale) per epoch for the from-scratch house price network.

Table 5: House price: from-scratch DL model results (dollars), test set

Model	RMSE	MAE	R^2
From-scratch DL	1,877,302	293,579	-1.1044

4.7 4-model comparison

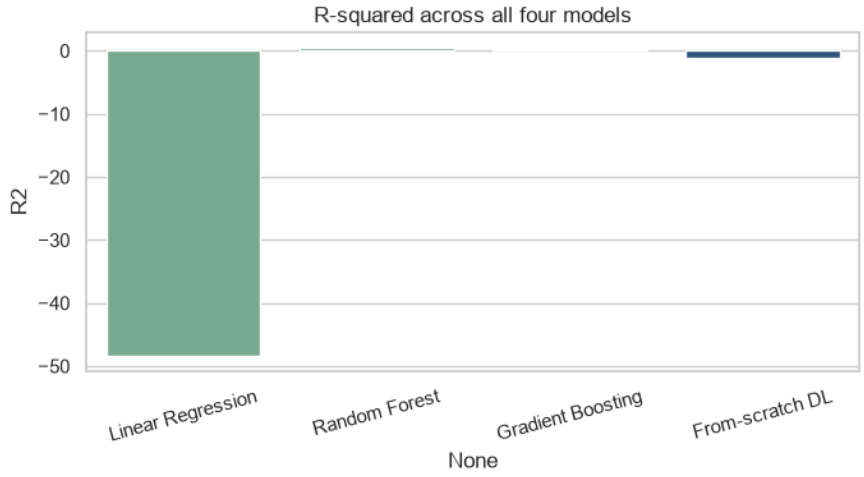


Figure 9: R^2 across all four house price models.

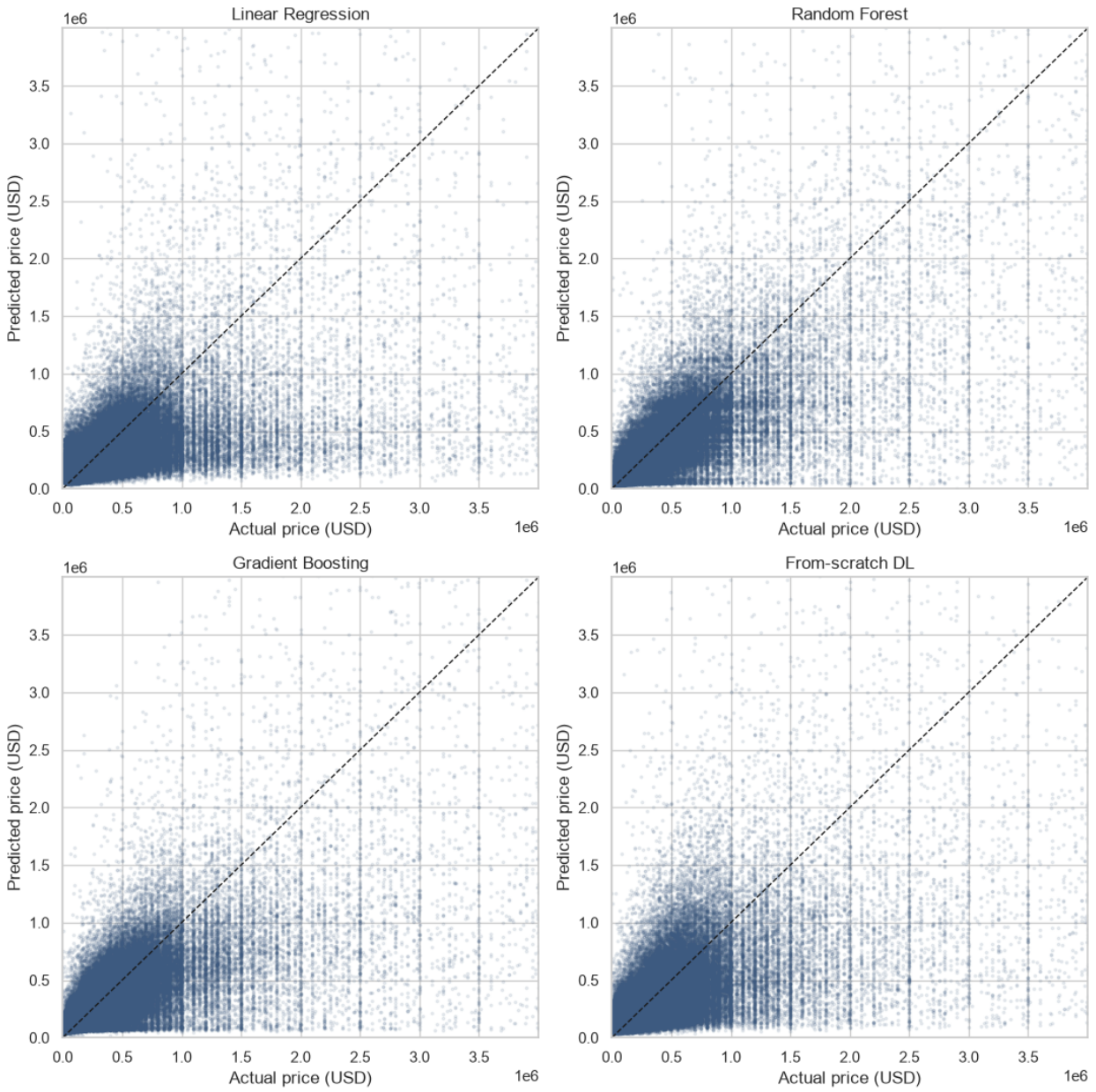


Figure 10: Predicted versus actual price, all four models, shared dollar axis.

Random Forest wins on every metric (RMSE 1,004,048; MAE 225,937; R^2 0.3980), ahead of Gradient Boosting (R^2 0.2397). The from-scratch network places third ($R^2 = -1.1044$), and Linear Regression finishes last by a wide margin ($R^2 = -48.3632$). That last number is the one worth stopping on: on MAE alone Linear Regression does not look catastrophic, it is only R^2 that reveals how badly it is actually doing.

The reason traces back to how heavy-tailed price itself is: the training data has a median of roughly \$315,000, a 99th percentile around \$4.2 million, and a maximum of \$875 million. Training on log-price tames that skew for fitting purposes, but does not remove the tail, it just moves where the tail causes damage. Every prediction is run back through `expm1` to turn a log-price back into dollars, and `expm1` is a convex, explosive function: a modest error in log-space, applied to a listing whose true price is already in the tens of millions, turns into

an enormous error in dollar-space, and R^2, built from squared error, is dominated by whichever handful of rows have the largest dollar-space misses.

What separates the two tree ensembles from Linear Regression and the from-scratch network is not smaller log-space error so much as a structural limit on how wrong a single prediction can get. A tree's prediction for any row is an average of training log-prices from whichever leaf that row lands in, so it can never output a value far outside the range its training leaves actually contained. Linear Regression has no such ceiling, its output is an unbounded linear combination of the input features, and the from-scratch network's output layer has exactly the same structure, a plain linear combination with no squashing activation, since nothing here calls for one the way sigmoid was needed for the classification tasks. That the network still beats Linear Regression by a wide margin on all three metrics, despite sharing this structural weakness, indicates the two ReLU hidden layers are doing real work, fitting the bulk of the log-price distribution with a piecewise-nonlinear function rather than a single flat plane, even though neither model gets the tree ensembles' built-in protection against an occasional wild extrapolation.

5 Customer Behaviour: Binary Classification

5.1 Problem description

This application predicts whether a Flipkart product review reflects a recommendation, defined as a rating of 4 or 5 stars, from the review's text and the product's price. It is a binary classification problem in the shape Lecture 03 calls Application 3: text flows through a learned representation to produce a sentiment or recommendation signal.

5.2 Dataset

The dataset is `flipkart_reviews.csv` from the Kaggle dataset:

https://www.kaggle.com/datasets/niraliivaghani/flipkart-dataset

loaded as 363,261 rows across 5 columns (`Product_name`, `Price`, `Rate`, `Review`, `Summary`). Flipkart was deliberately chosen as a different e-commerce platform (general merchandise across 75+ categories) rather than reusing the Sephora review data from Assignment 02, to work with a genuinely different review vocabulary and product mix. After dropping rows with an unparseable rating, a missing `Summary`, or a missing price, the cleaned dataset is 361,233 rows (2,028 dropped, 0.56%).

5.3 Data understanding and cleaning

The `Price` column arrived corrupted: the source file's Rupee symbol did not survive export and left behind stray bytes that broke a direct numeric conversion for every single row (363,261 out of 363,261 rows failed to parse as a plain number before cleaning). A regex that strips everything except digits and the decimal point recovers a normal-looking retail price distribution once applied. `Rate` also loads as text rather than a clean integer, with 44 rows carrying an unparseable or out-of-range value, dropped during cleaning.

Ratings pile up heavily at 5 stars (Figure 11), and the derived `is_recommended` target inherits that skew: 76.6% of reviews are labeled recommended. Reviews with a low rating also tend to run noticeably longer than five-star reviews, which fits how people write reviews in practice: a happy customer often just says "nice product," while a disappointed one tends to explain exactly what went wrong.

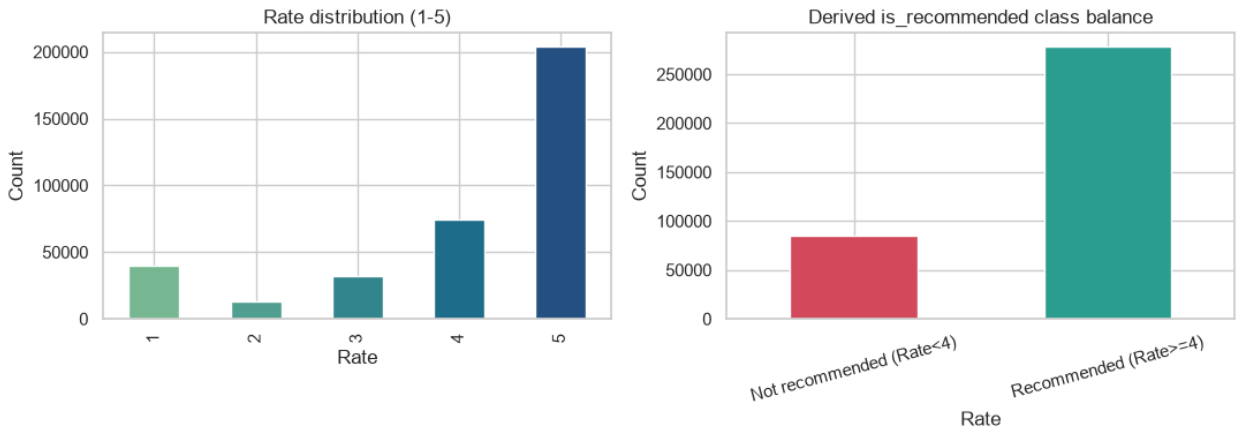


Figure 11: Rate distribution (1-5) and the derived `is_recommended` class balance.

The most consequential finding in this application's EDA concerned which text field to build the represen-

tation from. `Review` looks like ordinary free text at a glance, but is overwhelmingly a small set of one- or two-word stock phrases repeated tens of thousands of times each ("Wonderful" appears 18,567 times, "Awesome" 11,501 times, "Great product" 11,300 times), with only 3,842 unique review texts across 363,261 rows, 1.06%. `Summary`, by contrast, is almost universally present (99.44% of rows) and far richer, with 165,850 unique texts among the rows that have one, 45.9% (Figure 12).



Figure 12: Unique text as a percentage of rows, Review versus Summary. Review is overwhelmingly repeated boilerplate; Summary is far richer.

This gap matters beyond variety for its own sake. A model trained and evaluated on a text field this repetitive is not really tested on new language during evaluation, since most of what shows up in the test split is a word-for-word copy of something already seen in training, so the evaluation ends up measuring memorization of a small boilerplate vocabulary rather than genuine generalization. An earlier version of this notebook that used `Review` directly as the TF-IDF input scored close to 0.99 F1 across all four models, a result that was investigated directly once it looked suspiciously high: 99.1% of test-set `Review` texts turned out to appear verbatim in the training set. Switching the representation to `Summary` drops that train/test exact-text overlap to 59.7%, still non-trivial (short one-word summaries like "Good" or "Nice" still repeat a lot), but a real improvement rather than a cosmetic one, and this switch is the representation actually used in the results reported below.

A second, separate factor also keeps these scores high even after the Summary switch. For roughly three in four rows, the exact Summary text is at least 95% associated with a single label across every other row sharing that same text ("Excellent" is recommended 98.7% of the time it appears, "Very good" 96.2%, even a plain "Good" 88.2% of the time despite being the single most common Summary in the dataset). This is not evaluation leakage in the way the Review-text overlap was, it does not depend on the model having seen that literal string during training, it is a genuine property of how people write these summaries: someone who types "Excellent" is overwhelmingly likely to have also given 4 or 5 stars. The exception is informative too: "Ok" sits at only 62.5% purity, close to a coin flip, exactly what a genuinely ambiguous word should look like, which is reassuring evidence that the purity pattern reflects real sentiment-language correlation rather than a methodology problem.

A full-row duplicate check also found a large number of exact duplicate rows (identical product, price, rating, and text). These were kept rather than dropped: short generic reviews really do get written repeatedly by different real buyers of the same popular item, so collapsing them would throw away genuine signal about

how common that kind of review is, not just clean up parsing artifacts.

5.4 Representation

`Summary` text becomes a TF-IDF vector over unigrams and bigrams, chosen over `Review` for the leakage and vocabulary-richness reasons established above. Two separate vectorizers are fit, because the two model families need different vocabulary sizes: the classical models get the full vocabulary with no artificial cap, 416,770 unique terms, since scikit-learn's sparse linear models handle that size comfortably because the matrix stays sparse all the way through training. The deep learning model gets a vocabulary capped at 5,000 terms instead, roughly 1.2% of the full vocabulary, because a dense NumPy forward and backward pass has no sparse representation to lean on: every column becomes a real, materialized number for every row, so 416,770 of them would be far too slow and memory-hungry to train from scratch. This cap is a deliberate, stated trade-off, and it is also the most direct explanation for why the from-scratch model trails the classical ones in the comparison below: it is reading a small fraction of the vocabulary the linear models get to see. `Price`, once cleaned into a number, is standardized with a scaler fit only on the training split. `Product_name` stays out of the feature set entirely, it is high-cardinality free text (a small set of popular items dominate review volume, with most other products appearing only a handful of times each) that would mostly encourage memorization rather than generalization. `Review` itself is dropped rather than combined with `Summary`, for the same leakage reason: adding it back would reopen the memorization shortcut this section closed.

The 80/20 stratified split gives 288,986 training rows and 72,247 test rows. The classical feature matrix is 416,771 columns wide (TF-IDF plus price); the deep learning feature matrix is 5,001 columns wide.

5.5 Classical machine learning models

Logistic Regression, the model Assignment 02 deployed for this application, is kept as the linear reference point closest in structure to a single layer of the from-scratch network below. **Linear SVM** (`LinearSVC`, primal form since rows outnumber features) looks for the maximum-margin separating hyperplane rather than maximizing likelihood, a genuinely different objective that is a classically strong performer on high-dimensional sparse TF-IDF features. **Multinomial Naive Bayes** takes a completely different approach, estimating per-class word probabilities under a simplifying word-independence assumption; it trains in a fraction of a second and serves as a useful lower-bound baseline. Because Naive Bayes requires non-negative input and the standardized `Price` column can be negative, it is trained on the TF-IDF text block alone, a deliberate scope difference from the other two models, not an oversight. Trees (Random Forest, Gradient Boosting) were deliberately not chosen for this application: they are a poor fit for very high-dimensional sparse TF-IDF input, where each split only looks at one of thousands of near-orthogonal columns.

Table 6: Customer behaviour: classical model results, test set

Model	Accuracy	Precision	Recall	F1
Logistic Regression	0.9250	0.9279	0.9780	0.9523
Linear SVM	0.9295	0.9344	0.9765	0.9550
Multinomial NB	0.9035	0.8951	0.9900	0.9402

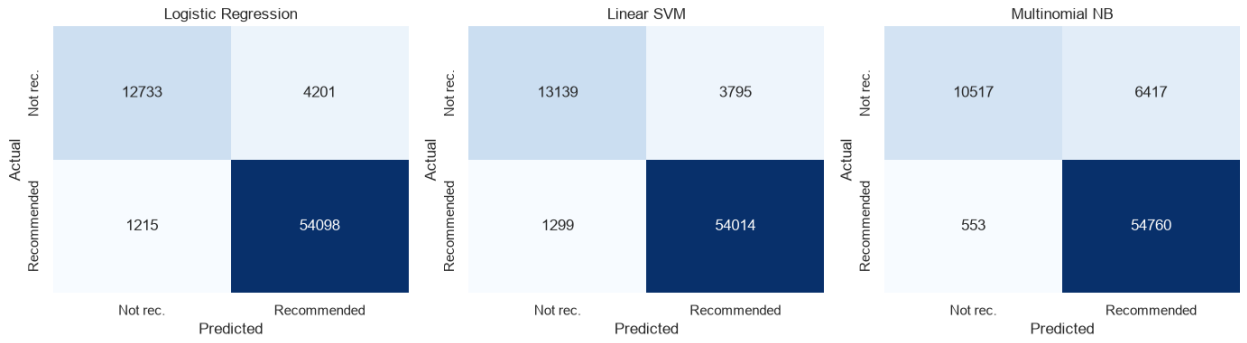


Figure 13: Confusion matrices for the three classical models on the customer behaviour test set.

5.6 From-scratch deep learning model

The network follows an architecture of $5000 \rightarrow 128 \rightarrow 32 \rightarrow 1$, with ReLU activations on both hidden layers, a sigmoid output, and binary cross-entropy loss. The first hidden layer is deliberately the widest of the three applications in this assignment relative to what follows it, since the TF-IDF input is sparse and high-dimensional (5,000 columns, mostly zero per row), and the first layer needs enough capacity to compress that information usefully rather than immediately bottlenecking it. One practical detail is worth stating explicitly: the input matrix is a scipy sparse matrix, and plain NumPy cannot operate on sparse matrices directly, so training uses mini-batch gradient descent specifically so that only one batch at a time needs to be densified, rather than converting the entire 288,986-row training matrix to a dense array in one shot (which would need several gigabytes of memory just to hold it). Each epoch shuffles the training rows and walks through them in batches of `BATCH_SIZE = 1024`, one forward pass, one backward pass, one gradient descent update per batch, so within a single epoch the weights update many times rather than once. `EPOCHS = 25` and `LEARNING_RATE = 0.1` (higher than the full-batch applications, since mini-batch updates happen far more often per epoch) were chosen as a practical balance between letting the loss visibly flatten out and keeping the runtime reasonable for a from-scratch NumPy loop over 291,000 training rows.

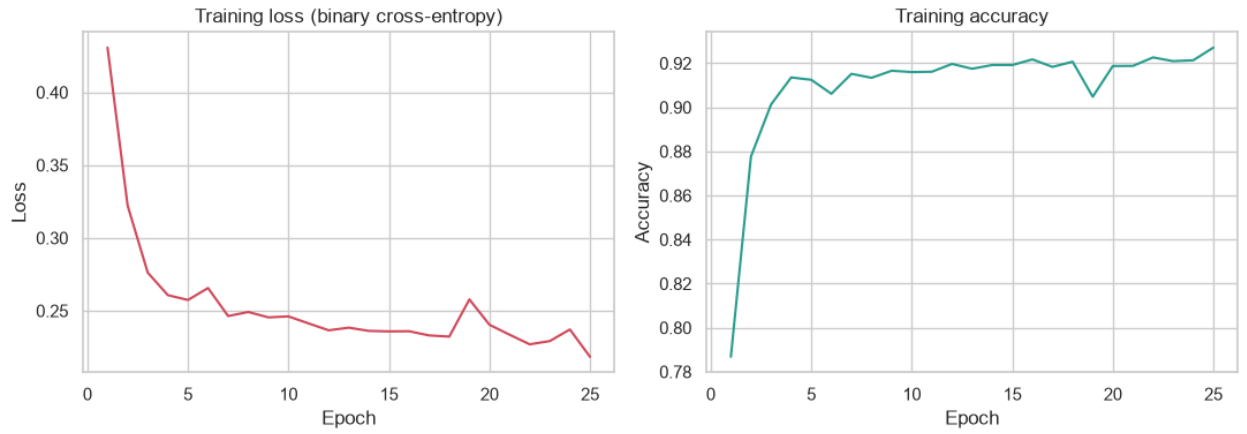


Figure 14: Training loss (binary cross-entropy) and training accuracy per epoch for the from-scratch customer behaviour network, evaluated on a fixed 20,000-row training subsample each epoch to avoid a full extra forward pass purely for the curve.

Table 7: Customer behaviour: from-scratch DL model results, test set

Model	Accuracy	Precision	Recall	F1
From-scratch DL	0.9221	0.9238	0.9790	0.9506

5.7 4-model comparison

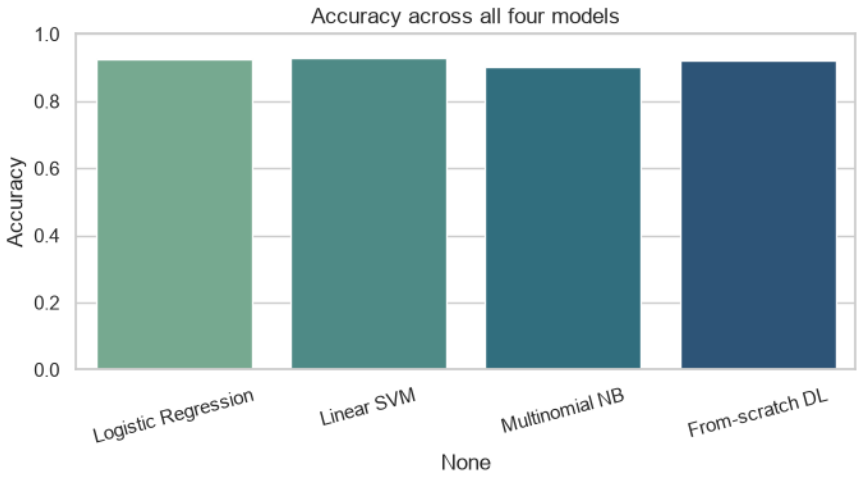


Figure 15: Accuracy across all four customer behaviour models.

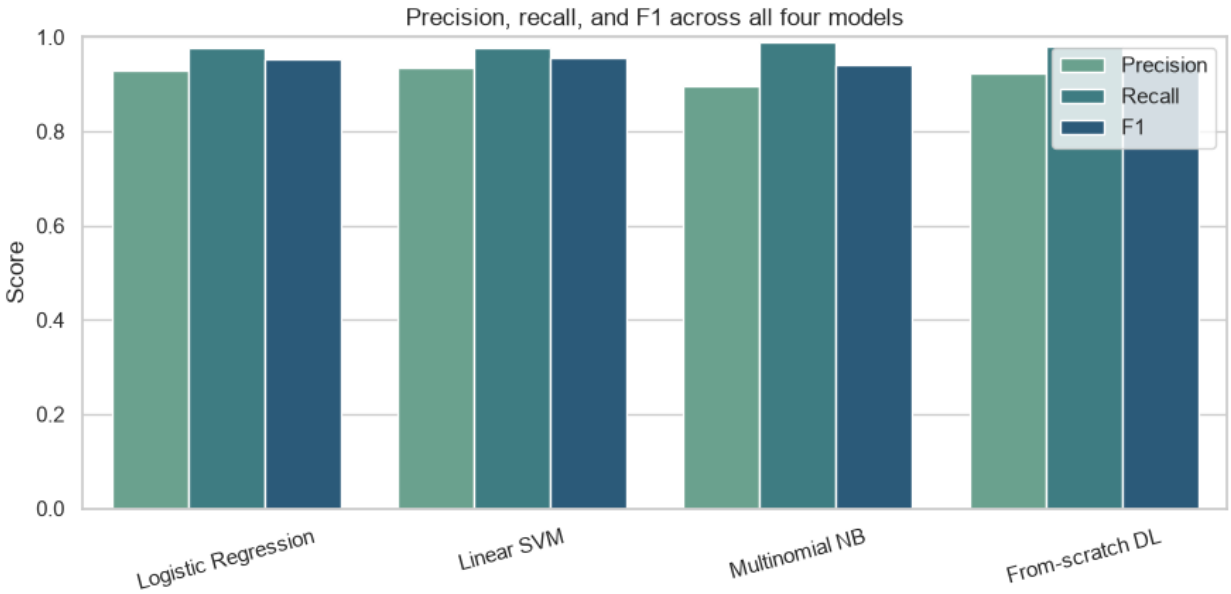


Figure 16: Precision, recall, and F1 across all four customer behaviour models.

Linear SVM comes out slightly ahead by F1 at 0.9550, with Logistic Regression close behind at 0.9523 and the from-scratch deep learning model right there with them at 0.9506. Multinomial Naive Bayes trails at

0.9402, though "trails" is relative, every model here clears 94% F1. Two separate effects explain why these scores sit this high, and both are established in the EDA section above: the Review-to-Summary switch cut train/test exact-text overlap from 99.1% to 59.7%, so these numbers are not mostly measuring memorization of boilerplate strings, and roughly three in four rows carry a Summary whose label is highly consistent across every row sharing that exact text, a real property of how sentiment language correlates with rating in this dataset rather than a flaw in the evaluation.

Within that context, the three models built on a linear decision boundary over the TF-IDF features, Logistic Regression, Linear SVM, and the from-scratch network's effectively-linear first layer before any nonlinearity is applied, land within half a point of each other. That closeness suggests the underlying task is close to linearly separable even on the richer Summary text, once a review carries a handful of strongly positive or negative words, and it is the same reason the deep learning model does not clearly beat the linear models: a nonlinear network's extra capacity only pays off when the true input-output relationship is not already close to linear, and a bag-of-words sentiment signal mostly is. Naive Bayes tells a different story: its recall is the highest of all four at 0.9900, catching almost every genuinely recommended review, but its precision is the lowest at 0.8951, since without the ability to weigh combinations of words against each other the way a linear model's coefficients can, it leans harder on individual strongly positive words being present at all, and it never sees the `Price` feature either, a second, smaller handicap. If anything is still holding the from-scratch network back from clearly beating the linear models, it is the 5,000-term vocabulary cap needed to keep training tractable, against the classical models' access to the full, much larger vocabulary; that the DL model still lands within half a point of Logistic Regression despite that handicap is a reasonable sign the from-scratch implementation is learning a sound representation.

6 Cross-Application Comparison

Table 8 summarizes all three applications by their primary metric (Accuracy for the two classification tasks, R^2 for the regression task), across all four models. Every cell is a transposition of the per-application results tables in Sections 3, 4, and 5, not new analysis.

Table 8: Cross-application summary: primary metric, all 3 datasets x 4 models

Application	Metric	Model	Score
Diabetes	Accuracy	Logistic Regression	0.7236
		Random Forest	0.7146
		Gradient Boosting	0.7149
		From-scratch DL	**0.7343**
House price	R^2	Linear Regression	-48.3632
		Random Forest	**0.3980**
		Gradient Boosting	0.2397
		From-scratch DL	-1.1044
Customer behaviour	Accuracy	Logistic Regression	0.9250
		Linear SVM	**0.9295**
		Multinomial NB	0.9035
		From-scratch DL	0.9221

Each application's three classical models differ, per the model choices justified in its own section above (Random Forest and Gradient Boosting for the two tabular tasks; Linear SVM and Multinomial Naive Bayes in place of tree models for the sparse TF-IDF text task), so this table lists each application's actual models rather than forcing them into shared generic column headers. Bold marks the winning score in each application.

The from-scratch deep learning model wins outright on one of the three applications (diabetes) and lands within roughly half a percentage point of the best classical model on a second (customer behaviour), while trailing clearly on the third (house price). This pattern, and why it breaks down the way it does, is the subject of the discussion below.

7 Discussion: What Changed from Assignment 02

7.1 The same three problems, a new model added

Every application kept the same problem framing as Assignment 02: diabetes binary classification, house price regression, and review recommend/not classification, each on a new dataset (a second BRFSS survey year for diabetes, a larger US-market sample for house price, a different e-commerce platform for customer behaviour). The genuinely new ingredient this round is the from-scratch deep learning model, evaluated against the same three classical models used before, but now as one of four models per application rather than the only comparison point.

7.2 What got harder: training time with no framework to lean on

Training a dense feedforward network with a pure-NumPy loop, at hundreds of thousands of rows, with no GPU and no vectorized-framework tricks beyond what NumPy itself provides, is noticeably slower and more fragile than calling `model.fit()` on a scikit-learn estimator. The diabetes and house price networks used full-batch gradient descent (the entire training matrix fits comfortably in memory as a dense array), while the customer behaviour network had to switch to mini-batch gradient descent specifically because its 5,000-column TF-IDF input, densified all at once across 288,986 rows, would need several gigabytes of memory just to hold. Getting a stable learning rate for each application also took real, direct benchmarking rather than a single default choice: house price's network suffered a dead-ReLU collapse at a learning rate of 0.3 before 0.01 was found to train stably, and diabetes's rate was chosen by comparing candidate rates directly rather than guessing.

7.3 Did the from-scratch model actually beat the classical models?

The honest answer differs by application, and that variation is itself the useful finding. On diabetes, the from-scratch network won outright by F1 (0.4648, narrowly ahead of all three classical models once balanced weighting was applied to every model). On customer behaviour, it landed within half a point of the best classical model (0.9506 vs. Linear SVM's 0.9550) despite reading only 5,000 of the full 416,770-term vocabulary available to the classical models, a deliberately accepted trade-off to keep a dense NumPy forward and backward pass tractable, not a modeling failure. On house price, it placed third, ahead of Linear Regression but well behind both tree ensembles ($R^2 = -1.1044$ against Random Forest's 0.3980), because its output layer shares Linear Regression's unbounded linear structure, with no ceiling the way a tree's leaf-average prediction has, on a target whose raw-dollar space is extremely heavy-tailed. Neither the customer behaviour vocabulary cap nor the house price extrapolation weakness is a bug; both are known, explainable limitations, small-network-vs-tree-ensemble on a heavy-tailed regression target, and a deliberately narrower vocabulary vs. a full-vocabulary linear model, worth naming plainly rather than glossing over.

7.4 What building the optimizer by hand exposed

One finding is worth highlighting because it is specifically invisible when calling `model.fit()` on a packaged implementation: plain gradient descent (`W -= lr * dW`, no per-parameter adaptive scaling, no Adam, nothing a framework quietly handles) has no defense against a feature sitting on a wildly different numeric scale than the rest of the input. This showed up independently while building two of the three from-scratch networks in this assignment.

In diabetes, the `year` column was initially left as the literal 2019/2020 integer rather than mapped to a 0/1 flag. Left that way, it entered the feature matrix at a scale of roughly 2020, dwarfing every standardized or small-integer feature beside it, and blew up the forward pass: the sigmoid overflowed at epoch 1, with a recorded loss of 17.49. Mapping `year` to a binary flag (2020 vs. 2019) fixed this completely, and is also the correct encoding on its own merits for a field with only two possible values, where a second one-hot column would just be a redundant mirror of the first.

In house price, `city_freq` (a raw count that runs into the thousands for popular cities) would have gone into the feature matrix unscaled, next to one-hot flags and already-standardized fields, the same class of problem. This one was caught by inspection before the network was ever executed, and fixed by folding `city_freq` into the same `StandardScaler` step as the other continuous columns, after first applying `log1p` to tame its own heavy tail (the same fix independently needed for `acre_lot` and `house_size`, both heavy-tailed counts whose raw scale produced double-digit standardized z-scores).

In both cases, every classical model trained on the exact same feature matrix was entirely unaffected: scikit-learn's solvers and the tree ensembles are either scale-invariant (trees split on raw thresholds regardless of scale) or considerably more robust to ill-conditioning than a hand-written gradient descent loop with a fixed step size. This is the kind of failure mode a packaged implementation absorbs silently, sensible default preprocessing, adaptive optimizers, or internal gradient clipping, none of which existed here to catch it, so it was only discoverable by building the optimizer by hand.

8 Conclusion

This assignment extended three intelligent systems, diabetes risk screening, house price prediction, and e-commerce review classification, each on a genuinely larger dataset than Assignment 02, with a fourth model per application: a feedforward neural network whose forward propagation, loss, backpropagation, and gradient descent update were all derived by hand in NumPy, with no autograd and no deep learning framework. The from-scratch implementation proved competitive rather than merely functional, winning outright on the diabetes task and landing within half a point of the best classical model on customer behaviour, while a clear, well-understood limitation, an unbounded linear output on a heavy-tailed regression target, explains its weaker showing on house price. Deriving every gradient by hand, rather than calling a packaged `model.fit()`, is what made it possible to state precisely why the network wins, ties, or loses on each task, and also surfaced a failure mode a packaged implementation would have silently absorbed: an unscaled feature is enough to derail full-batch gradient descent even when every classical model trained on the identical feature matrix is unaffected. The same representation-learning and optimization framework from Section 2 explains every one of these outcomes once the architecture, activation choice, and loss for each application are made explicit rather than left inside a library.

9 Reproducibility

9.1 Environment

All three notebooks were built and executed in a dedicated virtual environment at `assignment_03/.venv` (Python, registered as Jupyter kernel `assignment_03`), with the following key package versions: numpy 2.5.3, pandas 3.0.5, scikit-learn 1.9.1, scipy 1.18.1, matplotlib 3.11.2, seaborn 0.13.2, jupyter, nbconvert 7.16.6. No `tensorflow`, `torch`, or `keras` package is imported anywhere in the three notebooks. `RANDOM_SEED = 42` is fixed in every notebook for every split, shuffle, and weight initialization.

9.2 Re-running a notebook end to end

Each application's notebook is self-contained and can be re-executed from its own directory with:

```
../.venv/Scripts/python.exe -m jupyter nbconvert \
    --to notebook --execute --inplace \
    notebook/<app>.ipynb --ExecutePreprocessor.timeout=1800
```

replacing `<app>` with `diabetes`, `house_price`, or `customer_behaviour`. Each notebook ends by saving its trained artifacts to its own `model/` directory:

- `model_pipeline.joblib`: the best classical model, plus its fitted scaler/vectorizer.
- `feature_names.joblib`: the post-encoding feature column order.
- `input_schema.json`: documented input fields and target.
- `dl_weights.npz`: the from-scratch network's trained weights and biases.

followed by a reload-and-verify cell that reloads both artifacts and confirms their predictions match the in-memory versions computed during that same run. No API, web, or mobile layer is included in this assignment, so no additional run commands beyond notebook execution are required.